

EXPERIMENT NO: 6

Student Name: Parvinder Kaur

UID: 23BCS11591

Branch: BE-CSE

Section/Group: 23_BCS_KRG-1_B

Semester: 6th

Date of Performance: 28/02/2026

Subject Name: System Design

Subject Code: 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

Objectives

- Understand the architecture of a massive-scale media streaming system.
- Identify functional requirements such as direct-to-cloud uploads and personalized feeds.
- Identify non-functional requirements including CDN caching, high throughput, and latency constraints.
- Analyze the decoupling of synchronous API requests using event-driven stream processing.
- Design REST APIs for video management, search, and user interactions.

Procedure

1. Studied real-world video streaming architectures and content delivery networks.
2. Identified core entities such as Users, Videos, Interactions, and View Histories.
3. Analyzed the Pre-Signed URL pattern to bypass API Gateway bottlenecks for large file uploads.
4. Collected functional and non-functional requirements for media delivery.
5. Designed APIs for video processing, metadata retrieval, and social engagement.
6. Evaluated streaming protocols (HLS/DASH) and separated thumbnail storage for optimization.
7. Analyzed stream processing (Apache Flink/Kafka) for real-time, high-volume view aggregation.

Functional Requirements

1. Client should be able to create an account on the platform.
2. After successful login, client should be able to upload videos to the platform.
3. Client should be able to search for videos based on title, tags, or metadata. – ELASTIC SEARCH
4. Client should be able to watch videos in multiple different resolutions (360p, 480p, 720p, 1080p etc.) using Adaptive Bitrate Streaming. – IMPORTANT
5. System should support chunk-based video streaming using HLS and MPEG-DASH protocols.
6. Client should be able to like and unlike videos with real-time counter updates.
7. Client should be able to post and view comments dynamically with pagination support.

8. System should automatically encode uploaded videos into multiple resolutions for dynamic bandwidth handling.

Core Entities of the System

1. Users
2. User Metadata (Profile details)
3. Videos
4. Video Metadata (Title, Description, Thumbnail, Tags, Resolution, Status)
5. Video Segments (Chunks for HLS/DASH streaming)
6. Likes
7. Comments

4. API Endpoints

A. Client-Onboarding

1. User Registration: POST /api/v1/users/register
2. User Login: POST /api/v1/users/login
3. Update User Profile: PUT /api/v1/users/update

B. Video Upload

1. Initialize Video Upload (Pre-Signed URL): POST /api/v1/videos/upload/init
2. Complete Video Upload: POST /api/v1/videos/upload/complete
3. Get Video Details: GET /api/v1/videos/{video_id}

C. Searching & Video Playing

1. Search Videos: GET /api/v1/search?q={video_title}
2. Get Video Metadata: GET /api/v1/videos/{video_id}
3. Fetch HLS Manifest: GET /api/v1/videos/{video_id}/manifest.m3u8
4. Fetch DASH Manifest: GET /api/v1/videos/{video_id}/manifest.mpd

D. Like & Comment Handling

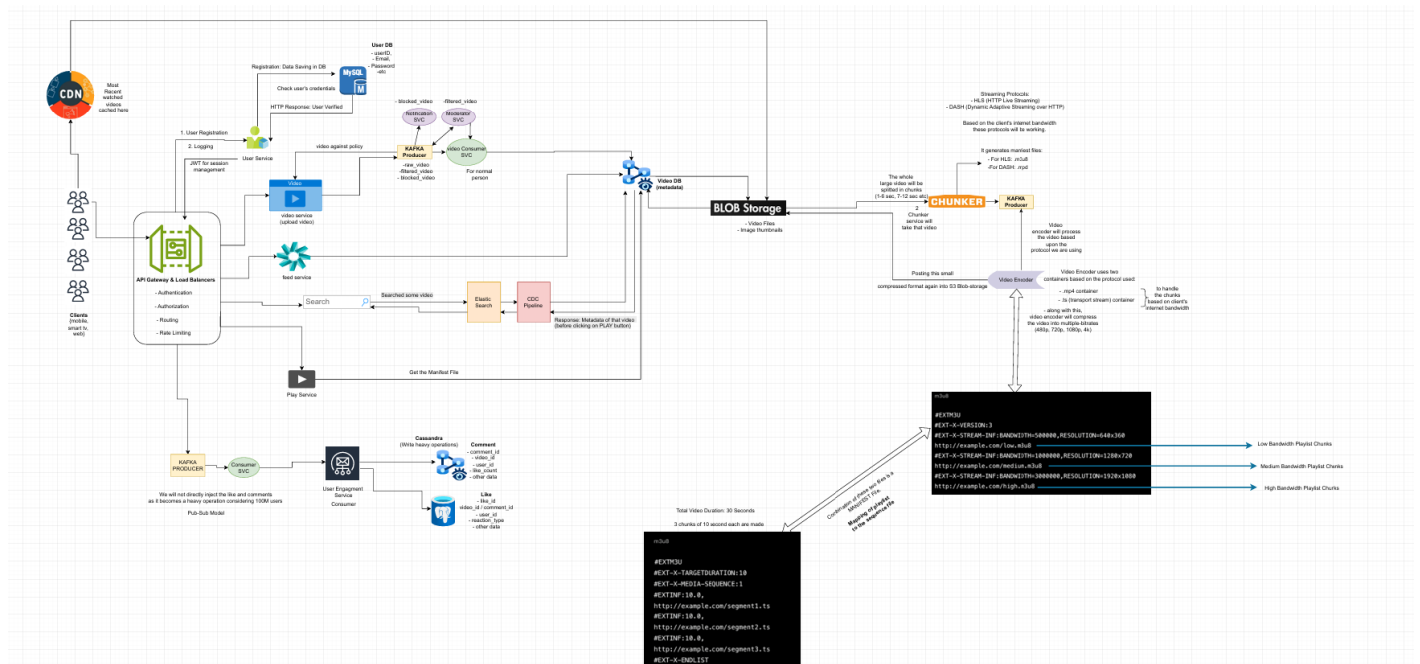
1. Search Videos: GET /api/v1/search?q={video_title}
2. Get Video Metadata: GET /api/v1/videos/{video_id}
3. Fetch HLS Manifest: GET /api/v1/videos/{video_id}/manifest.m3u8
4. Fetch DASH Manifest: GET /api/v1/videos/{video_id}/manifest.mpd

Non-Functional Requirements

1. **Scalability:** System should support millions of users and large-scale video storage with horizontal scaling.
2. **CAP Theorem: Availability >>> Consistency**
Availability is critical during video streaming.
Eventual consistency is acceptable for likes and comments.
3. **Latency:** 50–100 ms.
Client should experience zero or negligible buffering.
4. **High Throughput:** System should handle millions of concurrent video views and dynamic interactions.
5. **Fault Tolerance:** No single point of failure.
6. **Adaptive Bandwidth Handling:** System should dynamically switch video quality (360p, 480p, 720p, 1080p).

High Level Design (HLD)

The system consists of Client Applications, an API Gateway, and specialized microservices (Auth, Upload, Video Metadata, Feed, and Interactions). Heavy uploads bypass the backend via Pre-Signed URLs directly to Object Storage. Asynchronous transcoding pipelines process videos into HLS/DASH segments, which are delivered globally via a CDN. High-volume interactions are managed using Redis (Likes), NoSQL clusters (Comments), and a Kafka-to-Apache Flink stream processing pipeline for accurate view aggregation.



Learning Outcomes:

- Designed a highly scalable, event-driven video streaming architecture.
- Identified critical functional and non-functional requirements for media delivery.
- Designed REST APIs and decoupled asynchronous processing pipelines.
- Understood the application of CDNs, adaptive bitrate streaming, and stream processing for massive-scale systems.